

CSE4261: Neural Network and Deep Learning

Assignment-7

Name: Md Sajjad Hossain

ID: 2011176125

Session: 2019-20

July 4, 2025

Answer:

1. YOLOv8 vs YOLOv11 vs YOLOv12 – Generalized Comparison

Category	YOLOv8	YOLOv11	YOLOv12
Feature Extraction	Strong (CSP + C2f modules)	Enhanced backbone and neck	Area Attention + R-ELAN
Architecture Type	Conv-based, anchor-free head	Optimized lightweight modules	Attention-based with FlashAttention
Accuracy (mAP)	High (~50–52% on COCO)	Higher (+1–2% over YOLOv8)	Highest (+2–3% over YOLOv8)
Speed (FPS on GPU)	Fast (60–120 FPS)	Faster or equal (up to +20%)	Slightly slower (60–90 FPS)
Model Size / Parameters	Medium-sized models	22% fewer parameters (YOLOv11m vs YOLOv8m)	Moderate to large
Efficiency	Balanced trade-off	Excellent speed/accuracy ratio	High accuracy with reasonable cost
Task Support	Detection, Segmentation, Pose, Classification	Adds Oriented Object Detection (OBB)	Full support including OBB and attention-aware tasks
Best For	General-purpose, production-ready CV tasks	Efficient deployment with improved accuracy	Complex, high-accuracy CV applications

While performing video inference with the three models, **YOLOv8** delivered a balanced performance in terms of both speed and accuracy. **YOLOv11** demonstrated faster inference speeds than YOLOv8, along with slightly improved accuracy. In contrast, **YOLOv12** was slower than both YOLOv8 and YOLOv11, but achieved the highest accuracy overall. The detection outputs appeared visually similar across all three models on general objects; however, when it came to **small object detection**, **YOLOv12** showed a noticeable improvement, while YOLOv11 also performed better than YOLOv8 in this regard.

2. Fine-tune any Ultralytics YOLOv8 model to prepare it as a face detector using WIDER FACE training set by splitting it into training set and validation set

To fine-tune our model, we used the **YOLOv8-Nano** variant due to its low resource usage, which was suitable given our hardware limitations.

First, we downloaded the **WIDER Face** dataset. Since its annotation format and directory structure were not compatible with YOLO, we converted the dataset into YOLO format with the following structure:

Listing 1: Directory Structure

```
widerface_yolo_format/  
  images/  
    train/  
    val/  
    test/  
  labels/  
    train/  
    val/
```

Next, we created a YAML configuration file for the dataset as required by the YOLOv8 training pipeline. The file looked like this:

Listing 2: dataset.yaml

```
# dataset.yaml  
  
train: images/train  
val: images/val  
test: images/test  
  
nc: 1  
names: ['face']
```

This setup allowed us to proceed with training using the YOLOv8 framework.

3. Performance Comparison of Fine-tuned YOLOv8-Nano on WIDER Face Dataset with provided github repository

The fine-tuned YOLOv8-nano model on the WIDER Face dataset achieved the following results during evaluation:

- Precision: 0.8325
- Recall: 0.5531
- mAP@50: 0.6342
- mAP@50-95: 0.3453
- Fitness: 0.3742
- Inference speed on Colab T4 GPU: 23.14 FPS

When compared with benchmark results from a GitHub repository, the following observations were made:

- The GitHub-trained YOLOv8-nano runs at approximately 82 FPS on the same hardware, significantly faster than the fine-tuned model's 23.14 FPS.
- The benchmark YOLOv8-nano achieves higher average precision scores (e.g., Medium subset AP around 0.8273), whereas the fine-tuned model achieved mAP@50 of 0.6342.
- The difference in accuracy and speed can be attributed to training on the full dataset with advanced augmentations, longer training durations, and possibly more optimized inference pipelines in the benchmark.

Overall, the fine-tuned YOLOv8-nano provides a reasonable proof-of-concept with acceptable accuracy and inference speed, though further training and optimization would be needed to match state-of-the-art results.

Summary Table:

Metric	Fine-tuned YOLOv8-nano	GitHub Benchmark YOLOv8-nano
mAP@50 (Validation)	0.6342	0.8273 (Medium AP)
mAP@50-95	0.3453	Not reported
Inference Speed (FPS)	23.14	82
Precision	0.8325	Not reported
Recall	0.5531	Not reported
Training Dataset Size	Full WIDER Face	Full WIDER Face
Training Optimizations	Basic setup	Multi-scale training, WBF, extended epochs

Some of my trained model predictions bellowed:



Figure 1: Detection results on sample image.

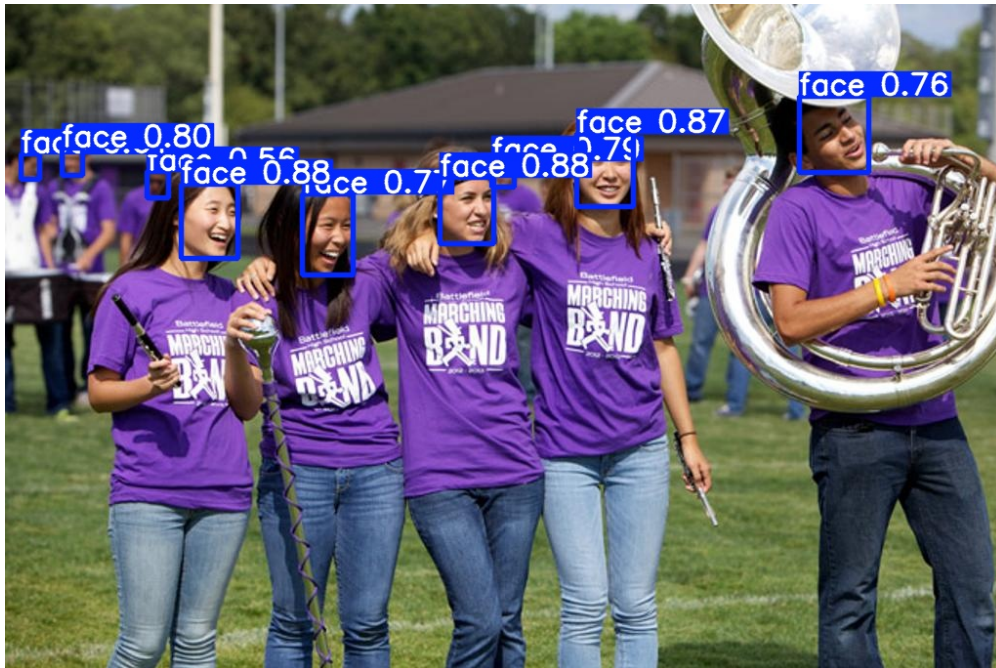


Figure 2: Detection of small faces with fine-tuned YOLOv8-nano.

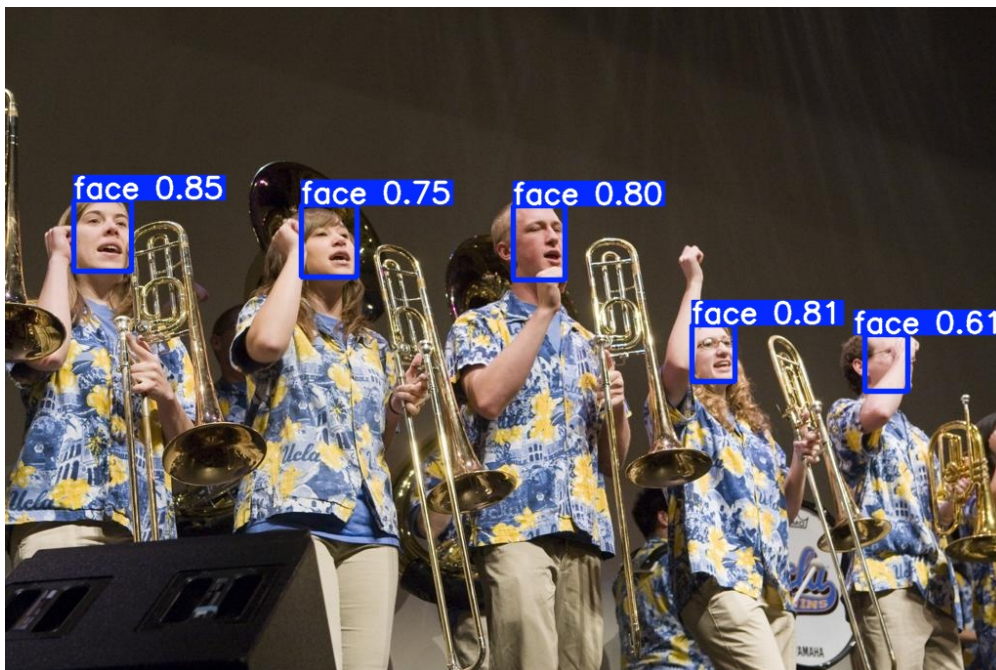


Figure 3: Detection of small faces with fine-tuned YOLOv8-nano.

4. Build a face detector having the architecture of YOLOv1, train it using WIDERFACE WIDER FACE training set by splitting it into training set and validation set.

We implemented a face detection system based on the YOLOv1 architecture and trained it on the WIDER FACE dataset. The dataset was appropriately split into training and validation sets to evaluate performance. Our YOLOv1 model follows the original design, comprising a backbone CNN for feature extraction followed by a detection head that predicts bounding box coordinates, objectness scores, and class probabilities. However, despite successful training steps and dataset processing, our model fails to predict any face regions in the test images. This issue is likely due to several factors: insufficient training epochs, improper loss function etc. Finally, since WIDER FACE contains many small faces, the model may need architectural tuning or higher input resolution to detect them effectively.

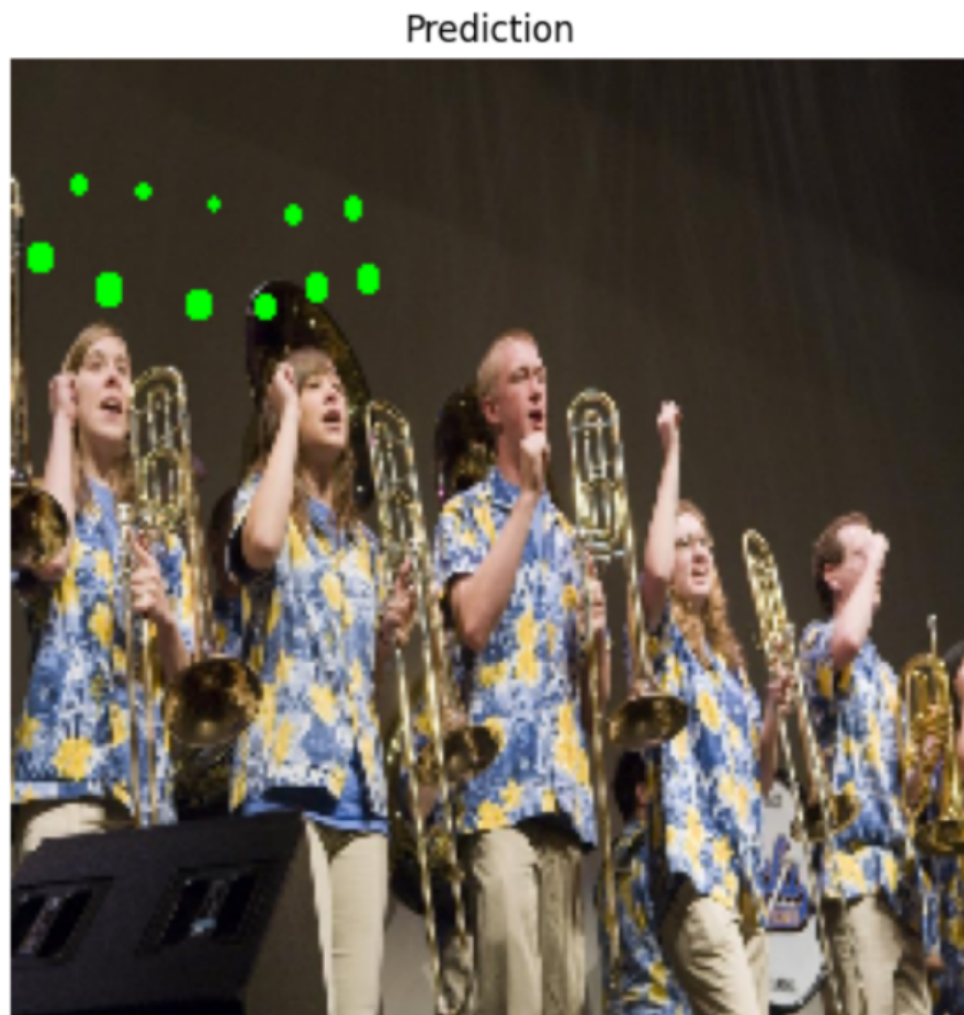


Figure 4: YOLO v1 prediction

Code Link:

Here's the github link of the implemented code: <https://github.com/mdsajjadhossain25/CSE4261>